

Documentación Técnica Preprocesamiento del Corpus

CODEFEST AD ASTRA 2026

Etapas: Construcción de la Base de Conocimiento

Equipo: Código Harpía

Miembros:

- ❖ Janeth Oliveros Ramirez
- ❖ Edgard Leonardo Castañeda García
- ❖ Javier Alejandro Penagos Hernández
- ❖ Nicolás Felipe Corredor Cortés

1. Arquitectura de la solución

El preprocesamiento separa la lectura (dependiente de la fuente) del procesamiento de texto (reutilizable). Cada formato del corpus tiene un lector que extrae texto crudo; un único procesador convierte ese texto en fragmentos (chunks) uniformes. Una función orquestadora conecta ambos por cada formato y entrega el resultado en JSONL, un chunk por línea.

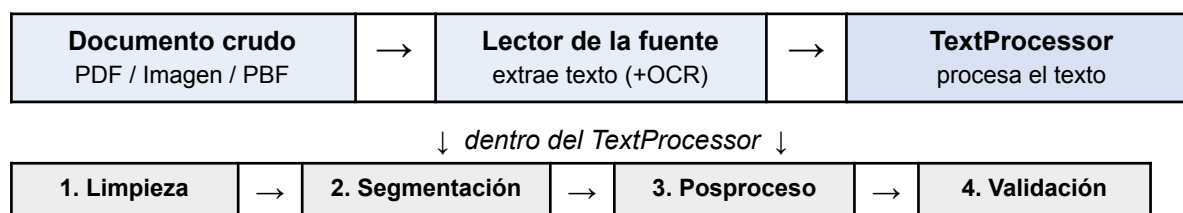
Capas

- ❖ Lectores (uno por fuente): PDFReader, ImageReader y la lectura de PBF. Abren su formato, extraen el texto con OCR de respaldo en PDF e imágenes, y decodificación de teselas vectoriales en PBF y devuelven texto crudo más metadatos (título, fecha).
- ❖ Procesador único (TextProcessor): recibe texto crudo de cualquier fuente y aplica limpieza, segmentación, posprocesamiento, validación de integridad y recuperación de texto perdido. Es independiente del formato de origen.
- ❖ Orquestadores (process_pdf, process_image, process_pbf): unen el lector con el procesador y devuelven los chunks en JSONL con los campos del contrato de salida (doc_id, chunk_id, fuente, formato, fenómeno, posición, num_tokens, num_palabras, texto, idioma, fecha, título).

Identificadores

Cada documento recibe un doc_id calculado como un hash de su ruta relativa dentro del corpus (desde la carpeta raíz «CORPUS CODEFEST AD ASTRA 2026» en adelante, con separadores normalizados). Al derivarse de la ruta relativa y no de la ubicación absoluta, el mismo archivo produce el mismo doc_id en cualquier máquina, y la ruta distingue documentos con igual nombre en carpetas distintas. Dentro de cada documento, los chunks se numeran de forma secuencial (posición 0, 1, 2, ...) y el chunk_id combina el doc_id con esa posición. Así, la unicidad del doc_id por documento más la secuencia interna garantizan que cada chunk_id sea único en todo el corpus.

Flujo general





Chunks en JSONL
un chunk por línea

El documento crudo pasa por el lector de su formato y luego por el TextProcessor, que aplica en orden las cuatro etapas internas (limpieza, segmentación, posprocesamiento y validación) y produce los chunks finales en JSONL.

2. Suposiciones adoptadas

- ❖ Solapamiento de chunks: los fragmentos se generan con una ventana deslizante que comparte hasta 80 palabras entre chunks consecutivos, para no perder contexto en las fronteras.
- ❖ Techos, no metas: cada chunk admite como máximo 250 palabras y 250 tokens; un chunk puede ser menor con tal de no cortar una oración.
- ❖ Completitud lingüística: la segmentación agrupa oraciones completas; nunca parte una oración salvo que una sola exceda por sí misma los límites.
- ❖ Idioma: se detecta automáticamente (es/en/pt). Como el segmentador no soporta portugués, ese caso se procesa con reglas de español, cuya puntuación es equivalente para segmentar.
- ❖ Conteo de tokens: se usa el tokenizador del encoder elegido, de modo que el conteo coincide con el que verá el modelo de embeddings.
- ❖ Identificadores reproducibles y únicos: se asume una carpeta raíz común del corpus para derivar rutas relativas; sobre esa base, el doc_id y los chunk_id son estables entre máquinas y únicos por documento (ver Sección 1, «Identificadores»).
- ❖ Imágenes: solo se conserva el texto extraíble por OCR (transcripción). Las imágenes sin texto útil se descartan mediante un filtro de confianza; no se generan descripciones.
- ❖ PBF (mapas vectoriales): los atributos de cada elemento se serializan como pares «atributo: valor» y se deduplican los elementos repetidos dentro de cada archivo.

3. Lógica de segmentación (paso a paso)

La segmentación transforma un documento crudo (con caracteres ruidosos, saltos de línea y basura de índice) en chunks limpios y completos. El proceso tiene cuatro etapas encadenadas:

1. Limpieza normaliza y quita ruido	2. Segmentación oraciones → ventana+solape	3. Posproceso recorta bordes cortados	4. Validación y recuperación de huecos
---	---	---	--

Etapas 1: Limpieza

Se normaliza la codificación (UTF-8/NFC), se eliminan caracteres de control, comillas dobles, marcadores de página (p. ej. «!12 !»), líneas de plantilla («Read more») y los puntos de relleno de índices («. . .»). Todo el texto se une en un párrafo continuo con espacios simples.

Etapas 2: Segmentación con ventana y solape

El texto limpio se divide en oraciones completas con un segmentador lingüístico.

Se agrupan oraciones en una ventana hasta acercarse a 250 palabras / 250 tokens, sin superar esos techos y sin partir ninguna oración.

El siguiente chunk retrocede sobre oraciones completas para solaparse hasta 80 palabras con el anterior, preservando contexto entre fragmentos.

Etapa 3: Posprocesamiento

Si un borde quedó cortado (un chunk empieza a mitad de oración o termina sin puntuación), se recorta el fragmento incompleto usando el chunk anterior como referencia, de modo que cada chunk empiece y termine en oración completa.

Etapa 4: Validación y recuperación

Se verifica que no se haya perdido texto entre chunks consecutivos y que se respeten los límites. Si un hueco real dejó palabras fuera, se crea un chunk adicional con ese texto y se reenumeran las posiciones.

Ejemplo trabajado: del texto crudo a 3 chunks

Texto crudo de entrada (con ruido típico de PDF):

```
!12 !
Contenido . . . . . 3
"La IA transforma la seguridad."
Read more
Los satelites vigilan el territorio.
Las capacidades espaciales dan ventaja.
Los sistemas autonomos plantean dilemas.
```

Tras la Etapa 1 (limpieza) queda un párrafo continuo, sin «!12 !», sin comillas dobles, sin «Contenido ... 3» de relleno ni «Read more»:

```
La IA transforma la seguridad. Los satelites vigilan el
territorio. Las capacidades espaciales dan ventaja. Los
sistemas autonomos plantean dilemas.
```

En la Etapa 2 se segmenta en oraciones y se agrupan con ventana y solape. Con fines ilustrativos se usa una ventana de 10 palabras y solape de 5 (en producción son 250 y 80); la lógica es idéntica: no se corta ninguna oración y cada chunk comparte su última oración con el siguiente.

Chunk 0	La IA transforma la seguridad. Los satelites vigilan el territorio.
Chunk 1	Los satelites vigilan el territorio. Las capacidades espaciales dan ventaja.
Chunk 2	Las capacidades espaciales dan ventaja. Los sistemas autonomos plantean dilemas.

El solape es visible: «Los satélites vigilan el territorio» aparece en los chunks 0 y 1; «Las capacidades espaciales dan ventaja» aparece en los chunks 1 y 2. Ninguna oración quedó partida, y las Etapas 3 y 4 confirman que los bordes están completos y que no se perdió texto.

4. CSV_Processor

Clase que procesa archivos .csv y .xlsx para extraer datos en forma de chunks con su respectiva metadata. El proceso sigue cuatro pasos:

Paso 1: Cargar el archivo y extraer sus metadatos. Se carga el archivo con pandas; según su extensión se usa read_csv (CSV) o read_excel (XLS/XLSX). A partir de la ruta se generan los metadatos base: un doc_id estable construido con un hash SHA-1 de la ruta de la fuente, el nombre del archivo como fuente, el formato, un título derivado del nombre y la

fecha de última modificación del sistema de archivos. Estos metadatos se asocian luego a todos los chunks generados.

Paso 2: Transformar cada fila en «unidades» de texto, respetando el límite de tokens.

Se recorre cada fila y columna construyendo unidades con el formato «columna: valor». Si una unidad supera max_tokens, se divide según su contenido:

1. Si contiene «|», se separa en elementos individuales.
2. Si es texto, se divide mediante puntuación y saltos de línea.
3. Si algún fragmento sigue siendo demasiado grande, se divide por palabras y, como último recurso, carácter por carácter (caso poco frecuente).
4. Cuando un valor requiere varios fragmentos, se marcan como continuaciones (columna (cont. 2), etc.) y cada unidad conserva su número de fila de origen.

Así se obtiene una lista de unidades de texto que luego se agrupa en chunks.

Paso 3: Agrupar las unidades en chunks finales con solapamiento. Las unidades se agrupan secuencialmente sin superar max_tokens. Cuando una nueva unidad excede el límite, se cierra el chunk actual y se inicia uno nuevo reutilizando las últimas unidades del anterior hasta alcanzar overlap_tokens (75 por defecto), conservando contexto entre chunks consecutivos.

Paso 4: Construir los registros finales y verificar la información. Cada chunk se convierte en un objeto JSON con su chunk_id, doc_id, texto, cantidad de tokens, idioma y metadatos del archivo. Finalmente se reconstruyen las unidades originales y se comprueba que todas aparezcan en al menos un chunk, generando un reporte de cobertura y de posibles unidades faltantes.

5. JSON_preprocessor

Contiene dos clases. JSONExtractor toma un archivo JSON, lo convierte en texto plano filtrando por la llave «body_text» y separando la metadata relevante. La clase Chunker toma el texto extraído y lo divide en chunks de máximo 250 palabras, procurando no cortar las oraciones. El proceso se resume en cuatro pasos:

1. **Lectura y extracción del JSON: JSONExtractor.extract().** Lee el archivo JSON y obtiene su contenido y metadatos principales.
2. **Conversión a texto plano: JSONExtractor._aplanar_json().** Convierte la información del JSON a texto, incluso si tiene una estructura compleja.
3. **Segmentación en chunks: Chunker.split() + pysbd.Segmenter.** Divide el texto en oraciones y las agrupa en chunks de máximo 250 palabras, evitando cortar oraciones.
4. **Generación de información de los chunks: Chunker.split() + AutoTokenizer.** Calcula palabras y tokens y agrega los metadatos (doc_id, fuente, fecha, título, idioma, etc.), dejando los chunks listos para generar embeddings.

Métodos auxiliares

_ruta_relativa(). Obtiene una ruta relativa del archivo a partir de la carpeta raíz, lo que permite guardar una referencia consistente a la ubicación del documento sin importar dónde esté almacenado localmente.

_asignar_doc_id(). Genera un identificador único (doc_id) con un hash SHA-1 de la ruta del archivo, para identificar cada documento de forma consistente y relacionar sus chunks con el documento original.

_detectar_idioma(). Detecta el idioma con langdetect; si el texto es muy corto o hay error, devuelve «desconocido». El idioma se usa luego en Chunker para separar oraciones con pysbd.

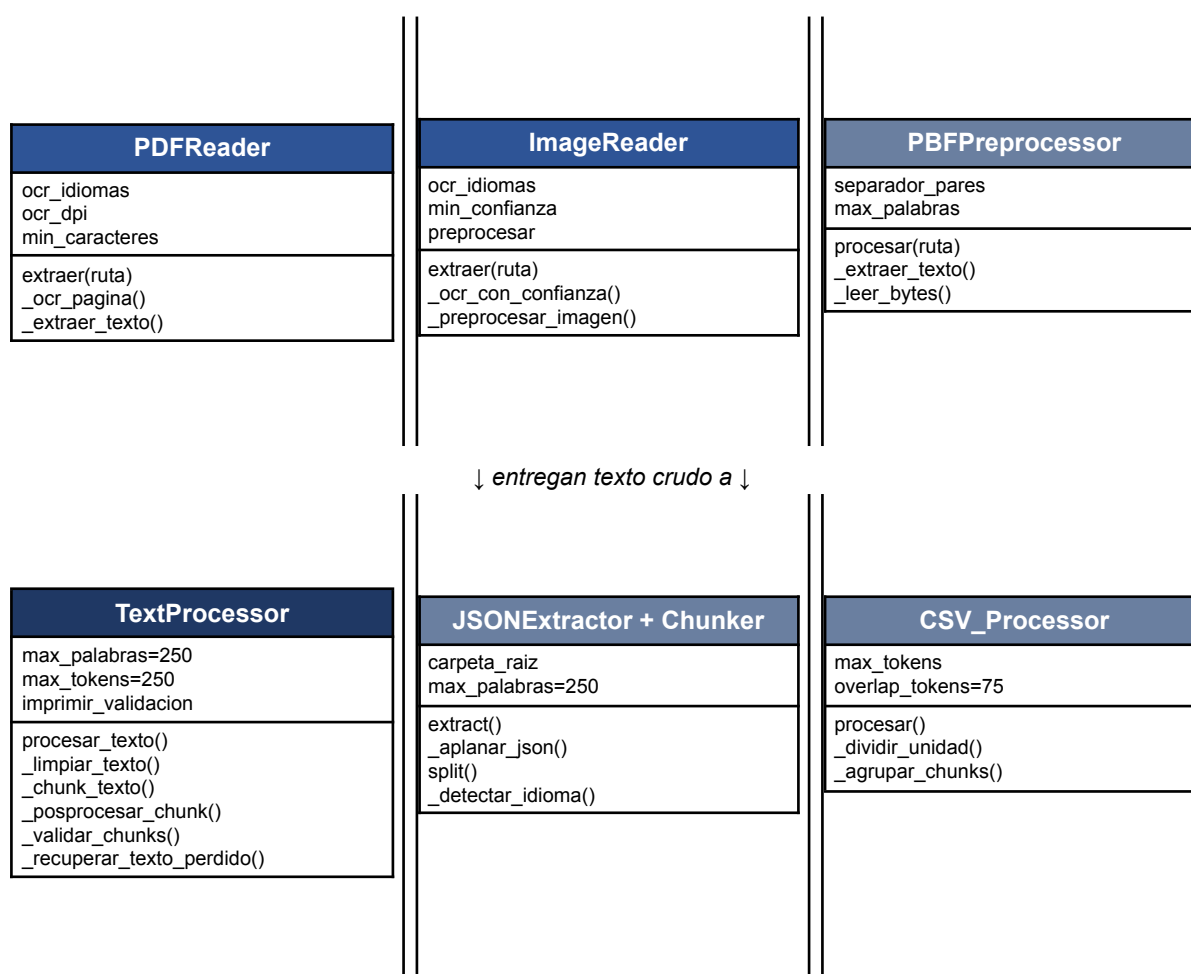
Funciones de salida

- ❖ procesar_json: retorna los chunks en una lista.
- ❖ procesar_jsonl: toma la lista de chunks y la retorna en formato JSON Lines.

6. Arquitectura en UML

Esta sección documenta las clases del preprocesamiento, sus responsabilidades y cómo se orquestan para obtener los chunks del corpus. El diseño sigue el patrón Lector + Procesador: cada formato tiene su lector/preprocesador, y todos convergen en la misma lógica de segmentación y en el contrato de salida (Tabla 1).

Diagrama de clases

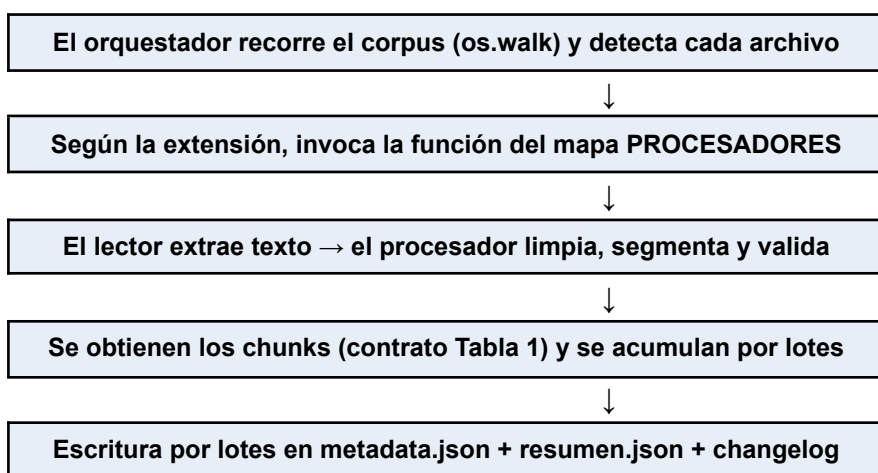


Los lectores (azul oscuro) solo extraen texto y lo entregan al TextProcessor, que centraliza la segmentación, el posprocesamiento, la validación y la recuperación. Los preprocesadores de JSON, CSV/XLSX y PBF (gris) encapsulan su propia extracción y aplican la misma estrategia de chunking (ventana de 250 con solapamiento). Todos producen registros con el mismo contrato de salida.

Proceso de obtención de los chunks

El orquestador recorre el corpus y, según la extensión de cada archivo, delega en la función correspondiente mediante un mapa formato → función. El resultado se acumula y se escribe por lotes en metadata.json.

Extensión	Funcion invocada	Modulo
.pdf	process_pdf()	PDFReader + TextProcessor
.json	procesar_jsonl()	JSONExtractor + Chunker
.csv / .xlsx	process_csv_file()	CSV_Processor
.pbk	process_pbf()	PBFPprocessor
.txt	procesar_txtl()	Text preprocessor
.jpg	process_image()	ImageReader + TextProcessor



El resumen y el changelog permiten reanudar el procesamiento sin repetir archivos ya completados, y validar que el conteo de chunks declarado coincide con las líneas escritas en metadata.json.

7. Selección del encoder

Para la generación de los embeddings se seleccionó el modelo intfloat/multilingual-e5-base, debido principalmente a su capacidad para representar información textual en múltiples idiomas, característica fundamental para garantizar que el sistema pueda procesar documentos y consultas independientemente de su idioma. Adicionalmente, el modelo está diseñado específicamente para tareas de generación de embeddings y recuperación semántica, lo que se ajusta directamente a los requerimientos del sistema, donde los fragmentos de texto deben transformarse en representaciones vectoriales para posteriormente realizar búsquedas de similitud mediante un índice FAISS.

El modelo emplea entrenamiento contrastivo, permitiendo generar representaciones en las que textos semánticamente relacionados presentan una mayor proximidad en el espacio vectorial, incluso cuando utilizan palabras o estructuras diferentes. Asimismo, genera vectores de 768 dimensiones, ofreciendo una representación semántica suficientemente rica manteniendo una dimensionalidad manejable para su almacenamiento y procesamiento. Finalmente, su soporte explícito para diferenciar entre consultas (query :) y pasajes (passage :), junto con su límite de 512 tokens por entrada, resulta adecuado para el esquema de chunking implementado en el sistema.

Este encoder además cuenta con la licencia MIT y está disponible en hugging face por lo tanto cumple con los requerimientos del reto y se ajusta a las necesidades del proyecto.